Egg segmentation and tracking

NOSH CV intern task: documented results extracted from the Colab notebook *NOSH_Egg_SAM.ipynb*

Links: GitHub repo: github.com/dhrupad1304/nosh-egg-segmentation | Tracking video (Google Drive): https://drive.google.com/file/d/1MYhljXmk2cjCzZFSpQ-AB04evRMmb1pN/view

1. Summary and status

Every number below comes from an output stored in the notebook, or, where marked, from the earlier chat log of the Mac experiments. Nothing was re-run or re-estimated for this document.

Task	Status	What the evidence supports
1. SAM reference masks	Done, with known defects	110 masks exist, but the set used for the headline scores has 10 masks covering over 80% of the image (inverted: background marked, not the egg). Further inverted masks exist that this rule misses (section 3).
2. Classical segmentation	Done	Experiment 7 (candidate generation, egg-likeness scoring, GrabCut refinement). Dev IoU 0.4636, eval IoU 0.5394 against the uncorrected references.
3. Evaluation and analysis	Done on the uncorrected references	Metrics, IoU histogram and worst-10 gallery exist. In the worst 10, 7 images have an inverted reference, so the scores understate the algorithm.
4. Video tracking	Partial	Detector finds the two eggs on most frames; reflections and ID churn remain (section 6).

Main finding. The reported IoU is dominated by reference-mask errors, not only by the algorithm. A corrected reference set is the single most useful next step; until then the scores are a lower bound on the segmenter's real accuracy, and they should be described as agreement with SAM pseudo-labels, not accuracy.

2. Dataset and split

The Roboflow export has 110 images (all 640 x 640) of eggs in varied scenes: in a hand, on a towel, in greens, on ash, on a keyboard, in boiling water, on foil. Roboflow's own train/valid/test folders are ignored. The project uses its own split, stored in *split.json* (it also records the grouping and seed used):

Split	Images
DEV (development)	88
EVAL (held out)	22
Overlap / missing	0 / 0 (verified in notebook cell 109)

Images come in bursts from the same scene, so the split is grouped to reduce leakage; the grouping method is recorded in the split file but its details are not printed in the notebook, so describe it from the code that created it.

3. Task 1: reference masks from SAM

Method. SAM ViT-B with the automatic mask generator (32 points per side, predicted-IoU threshold 0.90, stability threshold 0.92, minimum region 500 px). An egg-shape filter keeps a mask only if it covers 1 to 50% of the image, has solidity above 0.9, ellipse fit above 0.85 and touches at most one image border. Overlapping duplicates are removed, and masks smaller than 35% of the largest are dropped (code: *scripts/sam_reference_masks.py*). Images where the filter returned zero or several masks were reviewed, and a number were fixed by hand with box or point prompts (notebook cells 36 to 107).

Defects found. A check on the reference set used for scoring (notebook cell 155) found 110 masks, none empty, and 10 covering more than 80% of the image (images 133, 27, 45, 13, 124, 159, 78, 29, 132, 121). In these, SAM marked the background and left the egg as a hole. The first fix, inverting the mask (cell 158, images 27, 29, 78, 159), does not work: the original masks cover the egg as well, with only a thin outline missing, so the inverted mask is a ring plus noise.

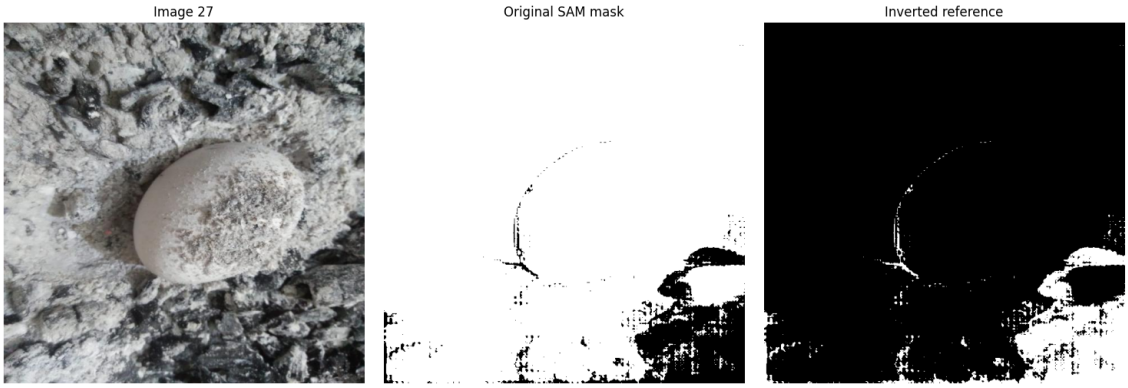


Figure 1. Image 27: the original SAM mask is white almost everywhere, including the egg; the "inverted reference" is black over the egg. Naive inversion does not recover the egg.

The worst-10 gallery (section 5) shows the problem is wider than the 10 flagged masks: images 2, 28, 95, 129 and 134 also have the background marked, yet cover under 80% of the image, so the area rule misses them.

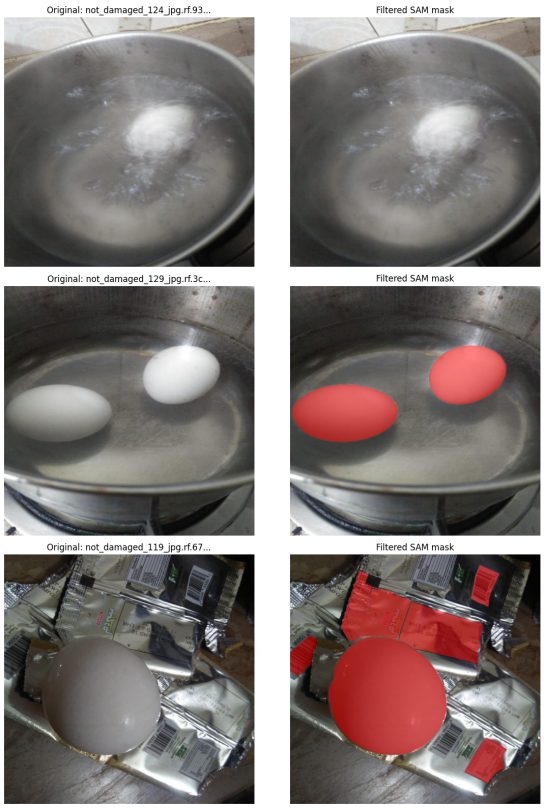


Figure 2. Spot check of the filtered SAM masks. Top: image 124 (egg half-submerged in boiling water) gets no mask. Middle: image 129 is correct. Bottom: image 119 has the egg plus two foil patches wrongly marked as eggs.

4. Task 2: classical segmentation

Final method (Experiment 7, *scripts/classical_segment_exp7.py*). For each image it builds a pool of candidate masks: Otsu thresholds on a Lab distance from the border colour and on the L, a, b and saturation channels (both polarities), plus GrabCut runs with 10%, 25% and 35% margins. Each candidate is cleaned (open 5x5, close 9x9, largest component). Candidates are scored on solidity, ellipse fit, boundary edge strength and colour contrast against a surrounding ring, minus a penalty for touching the image border. The top candidate is refined with mask-initialised GrabCut (eroded mask as sure foreground, dilated ring as probable background), then convex-hulled. The weights are in the script. Only the largest component is kept, so images with several eggs lose all but one.

Experiment log. The table is assembled from two sources: the Mac runs (earlier chat log) and the Colab notebook. The two use different reference masks, so rows from different sources must not be compared with

each other (the same script, exp4, scores 0.4711 on one and 0.5100 on the other). Experiment 1 was not recorded anywhere I could see.

Exp	Change	Dev IoU	Dev Dice	Reference masks	Outcome
1	First classical baseline	n/r	n/r	n/r	Not recorded
2	GrabCut, 10% margin rectangle	0.4719	0.5875	Mac masks	Superseded by exp 3
3	GrabCut, 25% margin	0.5445	0.6599	Mac masks	Best early method; margin slices eggs near the border
4	GrabCut + multiple contours	0.4711	0.5849	Mac masks	Discarded, worse than 3
4 (Colab)	Same script	0.5100	0.6154	Filtered masks (corrected)	Not comparable with Mac rows
5	GrabCut + central ellipse init	0.4404	0.5726	Mac masks	Discarded
5 (Colab)	Same script	0.4690	0.5883	Filtered masks (corrected)	Not comparable with Mac rows
6	Colour-seeded GrabCut (low saturation, high value)	0.3672	0.4789	Mac masks	Discarded. The function used was incomplete (no largest-contour step or return), so this result is unreliable
7 (dev)	Candidates + scoring + refined GrabCut	0.4636	0.5679	Original SAM masks (10 inverted)	Final method
7 (eval)	Same, held-out	0.5394	0.6449	Original SAM masks	Final, 22 images

Caution on exp 7 against exp 3 to 6. Exp 7 was scored on the uncorrected reference set, while the Colab re-runs of exp 4 and 5 used the filtered set. A fair comparison needs all of them scored against one corrected set. Parameters for each experiment are in the experiment scripts and in *results/experiments.csv*.

5. Task 3: evaluation and analysis

Metrics are per-image IoU and Dice against the SAM references, averaged (notebook cell 167, *scripts/task3_analysis.py*).

Split	Images	Mean IoU	Mean Dice	IoU >= 0.4	0.2 to 0.4	< 0.2	Ref. area > 0.8
DEV	88	0.4636	0.5679	50	14	15	9
EVAL	22	0.5394	0.6449	14	4	3	1

Failure categories follow a rough heuristic: reference area over 0.8 is flagged first, then IoU under 0.2, under 0.4, otherwise higher overlap. DEV and EVAL are different numbers; an earlier README listed them as equal by mistake.

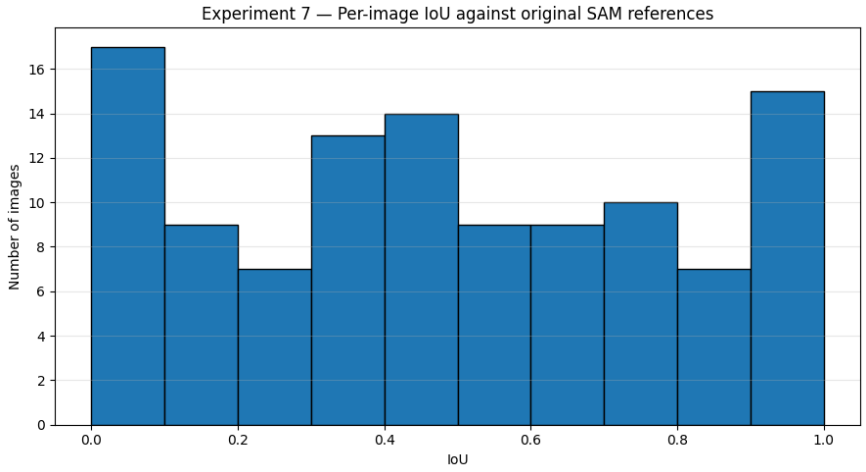


Figure 3. Per-image IoU. The distribution is bimodal: 17 images below 0.1 and 15 above 0.9.

Reading the histogram. A bimodal shape means the segmenter either finds the right region or misses it entirely; there is little middle ground. The pile at zero is the number to scrutinise, since the worst-10 gallery shows how many of those images have a wrong reference.

Worst 10 images, with the likely cause. Causes come from the gallery in Figures 4 and 5; reference area ratio is the fraction of the image the reference mask covers.

Image	Split	IoU	Ref. area	Pred. area	Cause seen in gallery
111	DEV	0.000	0.13	0.04	Algorithm: picked the dark crescent under the egg, not the egg
28	DEV	0.000	0.72	0.04	Reference inverted (background marked); prediction covers only part of the egg
106	EVAL	0.000	0.11	0.03	Algorithm: picked the black shadow shape beside the egg
45	DEV	0.000	0.87	0.09	Reference inverted; prediction is the egg
95	DEV	0.000	0.70	0.09	Reference inverted; prediction is the egg
42	DEV	0.001	0.05	0.04	Algorithm: picked a rod crossing an egg tray; reference is a partial egg
2	DEV	0.002	0.29	0.10	Reference marks the hand and leaves the egg as a hole; prediction is the egg
13	DEV	0.004	0.90	0.04	Reference inverted; prediction is the egg
129	DEV	0.011	0.70	0.06	Reference inverted; prediction finds only the right egg, misses the left egg
134	DEV	0.017	0.72	0.11	Reference inverted; prediction finds one of three eggs

What this shows. Seven of the worst ten (28, 45, 95, 2, 13, 129, 134) have a wrong reference, so their near-zero IoU is mostly a reference error. Three (111, 106, 42) are real segmenter failures, where it selected a shadow, reflection or rod instead of the egg. Separately, 129 and 134 show a real limitation: the algorithm returns one component, so multi-egg images lose the other eggs.

Where the algorithm genuinely fails. (1) Dark crescents and shadows beside white eggs score well on contrast and convexity and are chosen over the egg. (2) Only the largest component is kept, which loses eggs in multi-egg images. (3) The GrabCut margin variants cut eggs near the image border. (4) Low-contrast eggs (white on a white towel or in water) give weak edges and weak candidates.

Improvements, in order of expected value. First, build a corrected reference set (box or point prompts for the inverted images) and rescore every experiment on it. Second, keep all egg-like components instead of the largest one, scored per egg. Third, penalise candidates that are much darker than the image's brightest egg-like region (removes shadows), and drop the border margins. Fourth, report per-egg precision and recall alongside IoU.

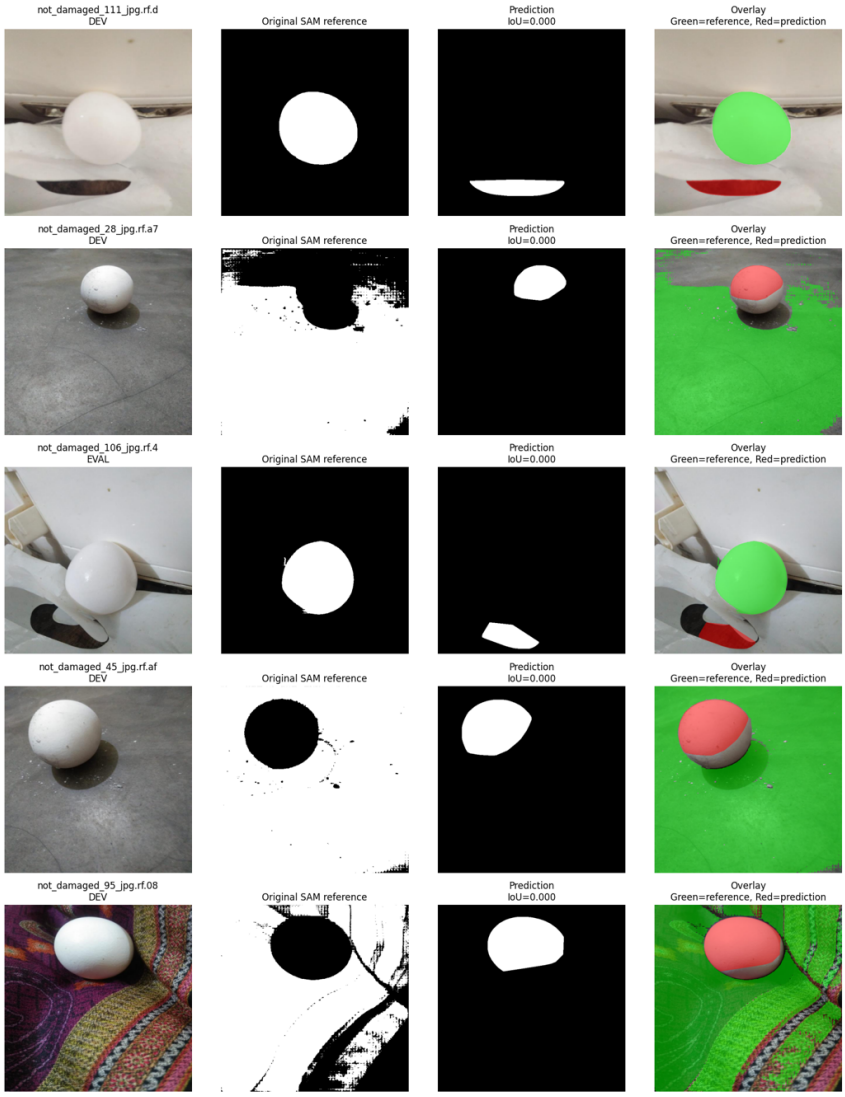


Figure 4. Worst-10 gallery, images 111, 28, 106, 45, 95. Columns: image, original SAM reference, prediction, overlay (green reference, red prediction).

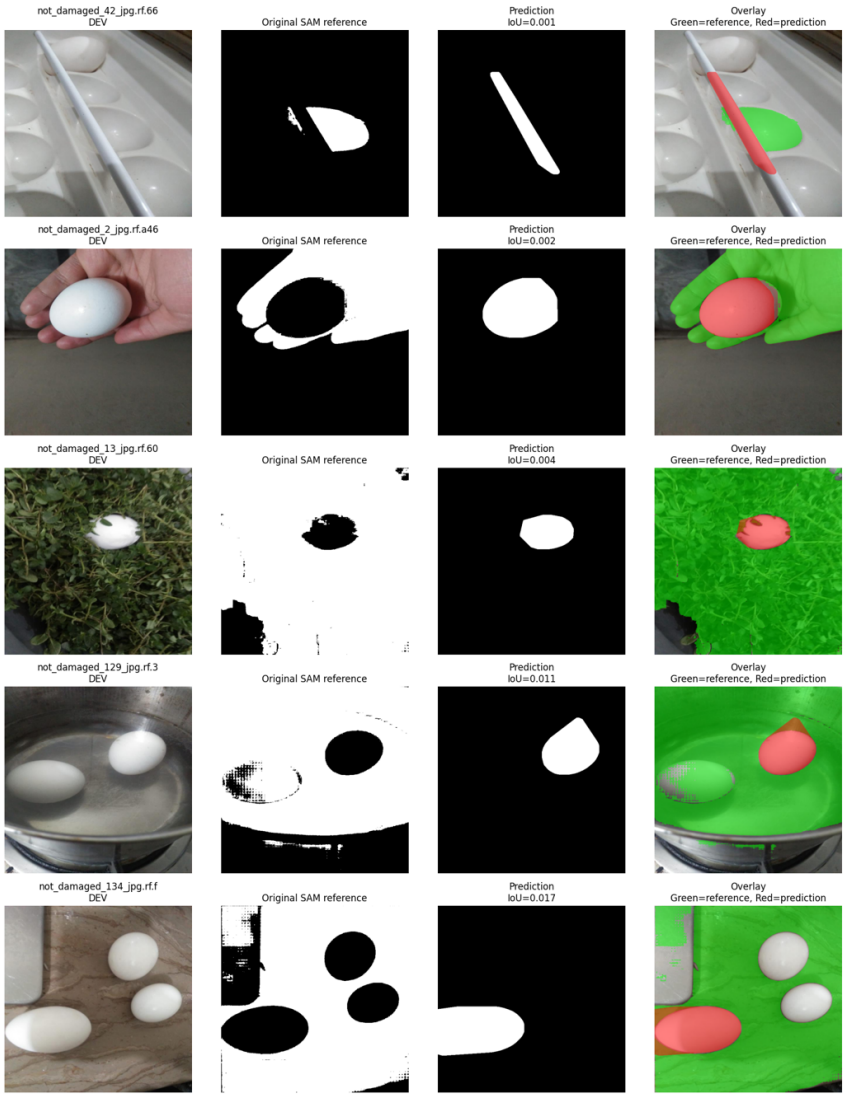


Figure 5. Worst-10 gallery, images 42, 2, 13, 129, 134.

6. Task 4: video tracking

Video. 60 s, 24 fps, 1440 frames, a fixed camera (a frame-to-frame motion proxy peaks at 0.42 and has median 0.01). Eggs appear between about 9.5 s and 41.5 s. Two eggs lie on a dark table and a hand moves them; the scene has a plaid shirt above the table, a title card with an arrow and text in the intro, a logo at bottom right, and glossy reflections under the eggs.

Detector (*scripts/task4_tracking.py*). Bright, low-saturation pixels (V at least 130, S at most 60); the top 38% of the frame and the logo corner are masked out. Opening with a 31 x 31 ellipse removes text, arrows and fingers (open before close; the reverse order merged the letters into blobs), followed by a 15 x 15 closing. Each connected component must pass an egg test (area 0.4 to 15% of the frame, solidity above 0.9, ellipse fit above 0.85, mean saturation under 70, brightness over 140) and a thickness test (distance-transform maximum above 40% of the shorter box side). A component merged with a dim patch is retried after an Otsu brightness cut. A dimmer, smaller blob directly under a larger egg is dropped as a reflection.

Tracker. Greedy nearest-centroid matching (distance under 150 px), at most two tracks, a track survives 60 missed frames and is drawn as OCCLUDED at its last position while unmatched.

Results. The final output video was inspected at 15, 20 and 24 s (Figure 6): at 15 s the right egg is tracked, but the left egg is missed and a label sits on its reflection; at 20 s the visible right egg is tracked and the egg under the finger gets no label; at 24 s both eggs are boxed, but the right egg is labelled Egg 4 although at most two tracks exist, so IDs are not stable. An earlier tracker version was audited over the whole clip (notebook cells 203 to 205): it produced six confirmed IDs (2, 3, 6, 7, 9, 10) for two eggs, with detections per frame of 0 on 527

frames, 1 on 367, 2 on 470 and 3 on 76, and 583 frames with an occluded track. The final version was not audited the same way.

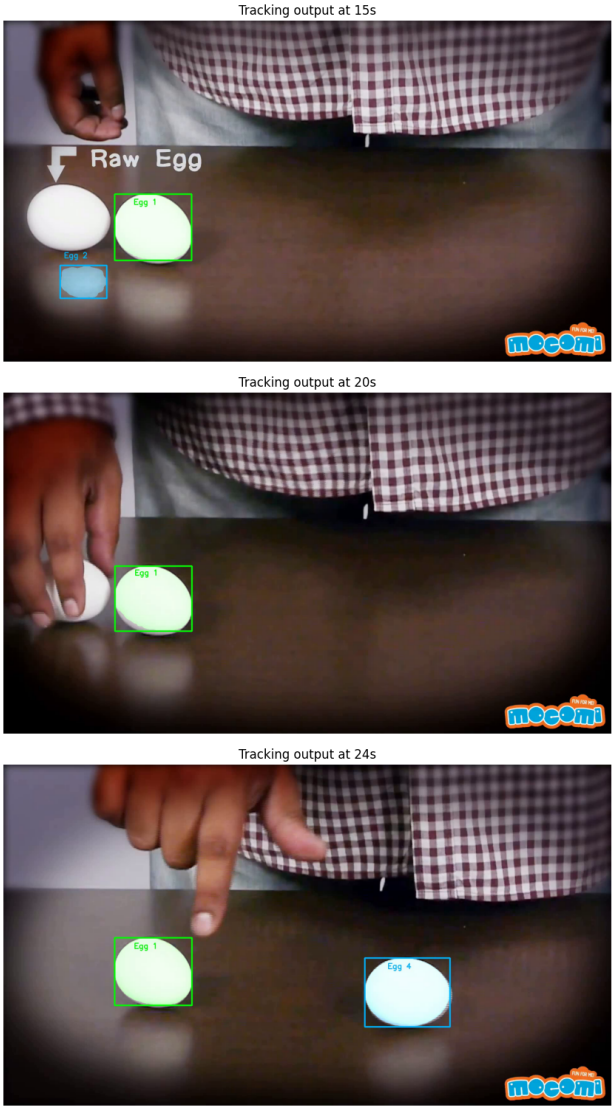


Figure 6. Final tracker output at 15 s, 20 s and 24 s.

Limitations to state. A hidden egg is not segmented while covered, only held at its last position. The reflection filter does not catch every reflection. ID numbers keep increasing after a track is replaced, so labels are not stable over the whole clip. The thresholds and the 38% crop are tuned to this video and assume a fixed camera. To improve: audit the final version frame by frame as the earlier one was, use IoU in the matching, keep IDs fixed by reusing freed slots, and filter the reflections by testing for a bright egg directly above.

7. Corrections to earlier outputs

Issue	Where	Correct value or handling
DEV and EVAL both reported as 0.5394 / 0.6449	First submission README and final_results.json (cells 136, 218)	DEV 0.4636 / 0.5679, EVAL 0.5394 / 0.6449. The corrected file is in results/final_results.json. The README inside the old ZIP still has the wrong table.
Exp 7 scored on a flawed reference set	Cells 134 and 146	Stated in every table; rescore on a corrected set when available.
Exp 4 and 5 numbers differ between Mac and Colab	Experiment table	Different reference masks, listed per row.
Exp 6 incomplete code	Chat log	Marked unreliable in the log.

8. AI tools disclosure (draft: edit to match what you actually did)

I used an AI assistant (Claude) for coding help and debugging, for drafting the experiment and analysis structure, and for reviewing screenshots of intermediate results. I ran all experiments myself in Colab and locally, chose which changes to keep or discard, and can explain each method: the Otsu and GrabCut candidates, the egg-likeness score, the morphological opening used to remove thin text, the distance-transform thickness test, and the centroid tracker.

Reproduction steps and the file list are in README.md.